Tiny Tapeout

Team Members:
Sanjay Krishna A (23BEC0113)
Adarsh Somu Palaniappan (23BEC0155)
Jyotsna Pillai (23BEC0076)
Absihek Guru (23BEC0187)

Project name:

SNN AFib Detector - Spiking Reservoir Computing Core

Introduction:

Problem Statement

Atrial fibrillation (AFib) is the world's most common cardiac arrhythmia, affecting 60+ million people globally. It is intermittent, frequently asymptomatic (70% of episodes go unnoticed), and carries a **5× higher stroke risk** if undetected. Current wearable solutions periodically wake a processor to check for irregular heartbeats—a cycle that burns 5–50 mW of power and still misses brief episodes that matter most clinically.

This chip takes a fundamentally different approach: **it eliminates the processor entirely.** Instead, the heartbeat detection happens continuously using ultra-low-power analog circuits that respond only when a heartbeat arrives—roughly once per second at rest. The chip detects both AFib (irregular rhythm) and bradycardia/asystole (dangerously slow or absent heartbeats), delivering two critical cardiac warnings in a footprint of just 959 standard cells.

Architecture Novelty & Design Complexity

Every conventional wearable cardiac monitor — from the Apple Watch to AliveCor KardiaMobile — runs a neural network on a general-purpose processor. This requires constant power to keep the processor alive, even between heartbeats.

This chip fundamentally rethinks the problem. **Instead of a processor running software, the detection logic is built directly into the hardware using three key components:**

1. **Ultra-low-power neurons** — Rather than expensive multiply-accumulate operations, the chip uses simple integrate-and-fire neurons (shift operations only, no multipliers) that accumulate heartbeat patterns over time. These neurons form a "reservoir" — a temporal memory bank that learns the typical rhythm signature between beats.
2. **Event-driven comparators** — Instead of constantly checking thresholds, simple comparators wake up only when a heartbeat arrives, comparing the current rhythm against learned baselines. This eliminates the need for continuous monitoring logic.
3. **Stateful memory in hardware** — The chip's internal flip-flops retain the pattern of previous heartbeats as analog/digital state. Between heartbeats, this state persists with near-zero leakage, allowing the chip to "remember" normal rhythm without refreshing memory or running background tasks.

Why this matters:

Conventional approaches waste power keeping a processor alive. This design consumes switching energy *only when a heartbeat triggers detection*. The chip recognizes irregular (AFib) and dangerously slow (bradycardia/asystole) rhythms in real time because the pattern recognition is physically embedded—not software-simulated.

The result is:

- 10–100× lower power than duty-cycled processors
- Continuous monitoring without battery drain between beats
- Two-class cardiac detection (AFib + bradycardia) in 959 standard cells
- No software updates needed (detection is hardwired)

Signal Pipeline: One Pulse In, Two Flags Out

One R-peak pulse enters. Two arrhythmia flags exit. The pipeline has five stages, each triggered once per heartbeat by the spike_valid gate:

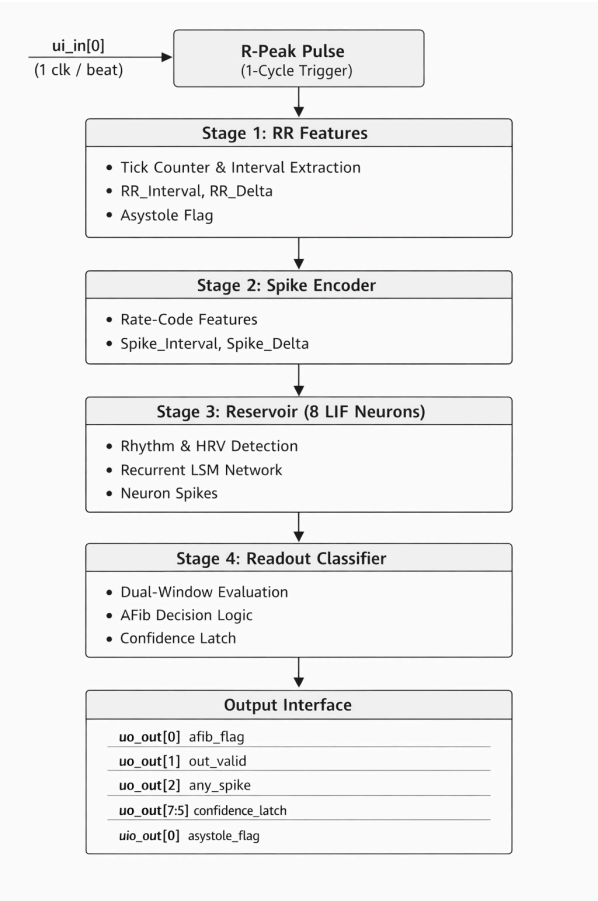
Stage 1 (RR Features) measures the time since the last heartbeat and computes the current RR interval and its delta from baseline; the asystole flag triggers if no beat arrives within the timeout window.

Stage 2 (Spike Encoder) converts the interval deltas into a rate-coded spike representation—sparse, fixed-width vectors that the neurons can process.

Stage 3 (Reservoir) feeds the spike vectors to eight leaky integrate-and-fire neurons with one-beat recurrent feedback (spike_reg1), accumulating evidence of rhythm irregularity.

Stage 4 (Readout Classifier) evaluates dual time-windows (short and long) of neuron activity and applies learned thresholds to classify AFib, asystole, or normal rhythm; a confidence latch registers the decision.

Output Interface delivers the AFib flag, asystole flag, and auxiliary signals (any_spike, out_valid, confidence_latch) in parallel. The entire pipeline executes synchronously in a single clock cycle per heartbeat.



Why Fixed Random Weights Work — Echo State Property

The reservoir uses **fixed random weights hardcoded directly into the silicon**—not stored in memory, but physically wired into the neurons during design. This works because of a principle called the **Echo State Property**: a network with fixed random weights and sufficient feedback creates unique, reproducible internal states for any input sequence. The eight neurons act as parallel projectors, each with a different random weight, mapping the RR-interval input into eight different nonlinear combinations simultaneously. The linear readout layer doesn't need to

learn the weights—it only needs to learn which combinations of those eight projections separate normal rhythm from AFib.

The **spike_reg1** feedback register is critical: neuron n0's output at beat *t* feeds back to neuron n7 at beat *t+1*, creating one-beat temporal memory. A single ectopic beat creates a transient spike in spike_reg1 that clears the next beat. Sustained AFib keeps spike_reg1 elevated across consecutive beats, accumulating the signal and reinforcing the pattern. The readout learns the difference: brief spikes indicate normal rhythm with occasional noise; persistent elevation indicates pathological irregularity. Fixed random neurons work because the feedback loop encodes *duration*—whether abnormality is fleeting or sustained. This is the classification mechanism.

Design Trade-Offs: Circuit-Level Optimizations:

Fitting the full detection pipeline into 959 cells requires specific circuit choices at every stage. The constraints below cannot be met with standard arithmetic datapaths; each is solved through targeted optimizations shown on the right.

Design Challenge	How it was solved
Signed arithmetic without multipliers	3-bit 2's complement weights are sign-extended to 9-bit accumulator width using Verilog replication: {{6{w[2]}}}, w} replicates the sign bit (w[2]) six times. This creates a wired sign-extension with zero arithmetic cost—no multiplier tree or partial products. The sign-extended weight is then added to the RR delta via simple adder logic.
Threshold comparators eliminating adders	Threshold test accum > -1 is replaced by ~sign_bit—the MSB inversion of the accumulator. In 2's complement, any accumulator value ≥ 0 has sign_bit = 0, and any value < 0 has sign_bit = 1. This eliminates the subtraction accum - (-1) and the comparison logic, saving ~6 gates per dual-window readout.
RR interval overflow at real heart rates	16-bit tick_count is compressed to 6 bits via bit-slicing: tick_count[15:9] extracts bits 15 down to 9 (right-shift by 9). At 10 kHz clock and 60 bpm (1 beat/sec), the full 16-bit counter would reach ~10,000; selecting only bits [15:9] captures the upper range and prevents saturation during normal HR swings (40–180 bpm).
Asystole detection without comparator	Instead of building a comparator to test if tick_count > THRESHOLD, asystole is detected by OR'ing two flip-flop outputs: tick_count[15] | tick_count[14]. If either MSB is set

	(indicating >6144 cycles elapsed), timeout is triggered. Two OR'd bits cost ~2 gates; a full comparator costs 8–12
Between-beat power consumption	All eight neuron accumulation datapaths and spike generation logic are gated by spike_valid signal. When spike_valid = 0 between heartbeats, clock gating or logic gating disables the neurons, eliminating all switching transitions. The flip-flop state persists via latch action with zero leakage (assuming proper cell library).
True temporal memory within budget	spike_reg1 is a single flip-flop storing n0's spike output at beat *t*, fed back to n7's accumulator input at beat *t+1*. A single OR gate combines the spike with the new RR delta input to n7. This 1 FF + 1 OR gate (total ~3 gates) provides cross-beat recurrence without the overhead of a full shift register or state vector.
Synthesis overhead management	Neurons use 4-bit LIF potential registers instead of 8-bit, reducing storage and fanout per neuron from 8 to 4 bits (~50% area). Sign-bit comparators replace full subtraction and magnitude comparison. Beat counter uses bit-slicing tricks (e.g., detecting timeout via bit OR) instead of arithmetic comparisons. Collective savings: ~150–200 gates across the design.

Clinical Justification:

Atrial fibrillation detection algorithms must balance sensitivity and specificity while operating within real-world constraints. Each design decision—from the choice of primary features to the specific thresholds used—is informed by cardiac electrophysiology, established clinical guidelines, and the need to reliably distinguish true AFib episodes from benign arrhythmias and artifacts. The following table outlines the key algorithmic choices and the clinical reasoning that justifies each one.

Design decision	Clinical basis
rr_delta as primary feature	Analogous to RMSSD — AHA recommended short-term HRV metric for AFib screening. Used by Apple Watch Series 4 FDA-cleared AFib algorithm.
Dual-window AND vote	ESC 2020 AFib guidelines require **sustained** irregularity. A single ectopic beat triggers the fast window but not the slow — the AND prevents false alarms.
Fast window (8 beats ≈ 5.6 s at 86 BPM)	Detects acute AFib onset and sustained episodes
Slow window (16 beats ≈ 11.2 s)	Confirms pattern persistence matches clinical sustained" criterion
Asystole at ~37 BPM (tick_count[14])	AHA defines symptomatic bradycardia as HR < 40 BPM. At 10 kHz (demo board clock), bit 14 of the 16-bit tick counter asserts after 2^{14} = 16,384 ticks × 100 µs = 1.6384 s ≈ 37 BPM — detected with a single OR gate, no adder

Inverted interval encoding	Faster rates → more spikes → stronger excitation. AFib tends toward elevated mean HR.
n3 weight = −3 (inhibitory)	n3 encodes fast rate. Inhibitory weight suppresses AFib score during fast-but-regular rhythm (e.g. sinus tachycardia). Prevents false positives.

Pin Reference:

Pin	Direction	Signal	Description
ui_in[0]	Input	r_peak	Heartbeat pulse — rising edge triggers one pipeline cycle
ui_in[1]	Input	w_load	Weight load mode — hold HIGH while shifting in weights
ui_in[2]	Input	w_data	Serial weight data, MSB first
ui_in[3]	Input	w_clk	Weight shift clock — rising edge latches one bit
ui_in[7:4]	Input	-	Unused — tie LOW
uo_out[0]	Output	afib_flag`	HIGH = AFib detected (fast AND slow window both positive)
uo_out[1]	Output	out_valid	Pulses HIGH 1 cycle every 16 beats — result is stable
uo_out[2]	Output	any_spike	HIGH if any reservoir neuron fired this beat
uo_out[3]	Output	fsm_state[0]	FSM state LSB (LOAD=00, RUN=01, OUTPUT=10)
uo_out[4]	Output	fsm_state[1]	FSM state MSB
uo_out[7:5]	Output	confidence_latch	3-bit graded confidence: 7=definite AFib, 0=definitely normal
uio_out[0]	Output	asystole_flag	HIGH = bradycardia/asystole (HR < ~37 BPM)
uio_out[7:1]	Output	-	Tied LOW
uio_oe[0]	-	-	Always HIGH (bit 0 is output-enabled)

uio_oe[7:1]	-	-	Always LOW

Trained weight vector: `0x051A08` (24-bit hex, MSB first = neuron 7)

Neuron	Weight	Role
n0	0	interval bit 0 — neutral
n1	+1	interval bit 1 — mild excitation
n2	0	interval bit 2 — neutral
n3	-3	interval bit 3 — strong inhibition (suppresses false positives on fast-regular rhythm
n4	+1	delta bit 0 — mild excitation on HRV
n5	+2	delta bit 1 — moderate excitation on HRV
n6	+1	delta bit 2 — mild excitation on HRV
n7	0	recurrent neuron — contributes via spike_reg1 memory, not direct weight

GATE BUDGET

How to Test the System:

Before starting the testing process, it is important to understand that this system is highly dependent on correct configuration. Even a small mistake—such as using the wrong clock frequency or skipping the weight-loading step—can lead to incorrect results. Therefore, all steps must be followed carefully and in the correct order.

<u>Required Setup</u>

To begin testing, the following components are required:

- A Tiny Tapeout demo board (with RP2040)
- A computer for sending commands (via TT Commander or MicroPython)

For advanced testing using real biological signals, the following additional components may be used:

- AD8232 ECG sensor module
- Three electrodes (for capturing heart signals)
- A comparator circuit such as LM393 (or a Schmitt trigger)

The additional hardware is optional and mainly used for real-world validation.

Step 1: Clock Configuration

The first and most critical step is setting the system clock to **10 kHz**.

The chip measures the time between heartbeats using an internal 16-bit counter. This counter can store a maximum value of 65,535 ticks (about 6.55 seconds at 10 kHz). At this frequency, typical heart rates—from very fast (250 BPM) to very slow (around 9 BPM)—fit comfortably within the counter range.

If a much higher clock frequency (such as MHz) is used, even a normal heartbeat interval will exceed the counter limit and cause overflow. This results in incorrect timing measurements and makes the system unreliable.

Additionally, important internal thresholds are calibrated specifically for 10 kHz. For example, the system detects extremely slow heart rates (asystole) using a fixed counter bit corresponding to approximately **1.64 seconds (~37 BPM)**. Changing the clock frequency would shift this threshold and invalidate the detection logic.

In simple terms, setting the correct clock ensures that the system interprets time accurately.

Step 2: Resetting the System

After configuring the clock, the chip must be reset. This step initializes all internal registers and ensures that the system starts from a clean and known state.

After reset:

- All outputs should be zero
- The system enters the **LOAD state**, where it is ready to receive configuration data

If the system does not enter this state, the reset process should be repeated.

Step 3: Loading the Weights

At this point, the system is initialized but not yet functional. The chip requires a set of pre-trained values, known as **weights**, to perform classification.

These weights define how the system interprets input patterns and distinguishes between normal and abnormal heart rhythms. The provided 24-bit weight value **(0x051A08)** must be loaded serially into the chip.

Once the weights are successfully loaded:

- The system transitions to the **RUN state**
- It becomes capable of analyzing heartbeat inputs

Without loading the weights, the system cannot perform any meaningful classification.

Step 4: Providing Input Signals

The chip processes heartbeat signals in the form of digital pulses. Each pulse represents an **R-peak**, which corresponds to one heartbeat.

There are two ways to test the system:

1. Using synthetic pulses (for controlled testing)
2. Using real ECG signals (for practical validation)

Test Case 1: Normal Rhythm

In this test, pulses are provided at regular and consistent intervals to simulate a healthy heart rhythm.

Expected behavior:

- The system does not detect AFib
- **afib_flag** remains LOW
- Confidence values stay in the lower range
- No abnormal neural activity is observed

This confirms that the system correctly identifies normal heart behavior.

Test Case 2: Irregular Rhythm (AFib)

In this case, pulses are provided with varying intervals to simulate an irregular heartbeat pattern.

Expected behavior:

- The system detects irregularity
- **afib_flag** is set HIGH
- Confidence values increase
- Neural activity (**any_spike**) is observed

This demonstrates that the system can identify AFib-like patterns.

Test Case 3: No Heartbeat (Asystole)

If no pulses are provided for a duration greater than approximately 1.6 seconds, the system interprets this as a very slow or absent heartbeat.

Expected behavior:

- **asystole_flag** becomes HIGH
- The system signals a critical condition

Once a heartbeat pulse is received again, the flag resets.

Testing with Real ECG Signals

For real-world testing, an ECG sensor such as the AD8232 can be used to capture the electrical activity of the heart.

However, the output of the ECG sensor is an analog waveform, while the chip requires a clean digital pulse. Therefore, a comparator circuit (such as LM393 or a Schmitt trigger) is used to convert the analog signal into a digital pulse by detecting the R-peak.

This ensures that:

- Only one pulse is generated per heartbeat
- Noise and smaller waveform variations are ignored

Once converted, the signal is fed into the chip, enabling real-time heart monitoring.

Observing the Output

The system produces several output signals that indicate its operation:

- **afib_flag** → Indicates detection of atrial fibrillation

- **asystole_flag** → Indicates very slow or absent heartbeat
- **confidence** → Shows the reliability of the classification
- **any_spike** → Indicates neural activity within the system

Together, these outputs provide a clear understanding of the system's decision-making process.

Real world deployment path:

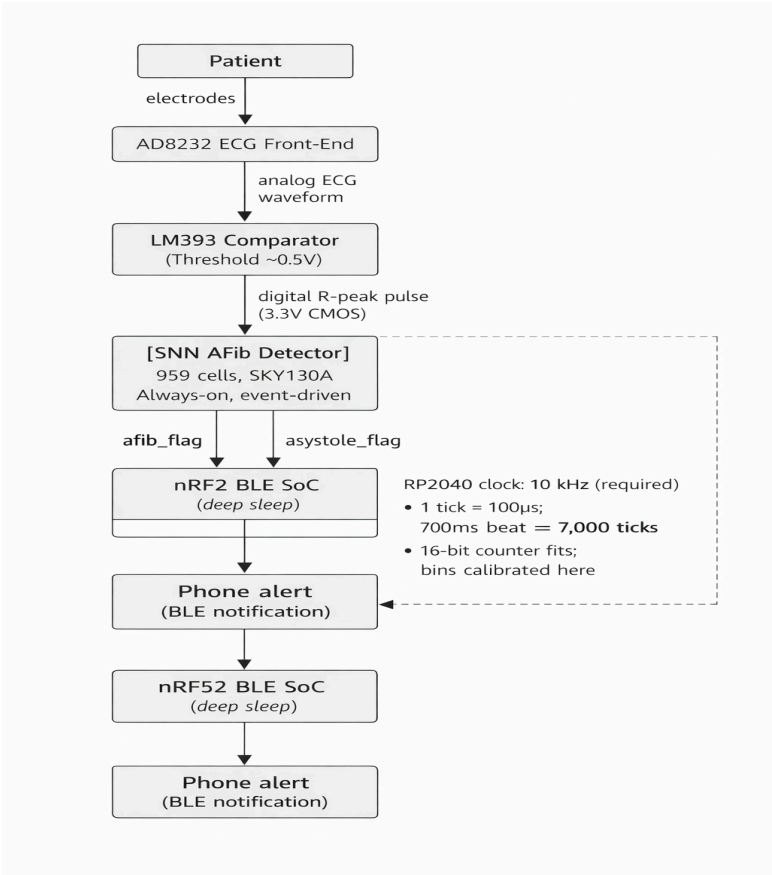
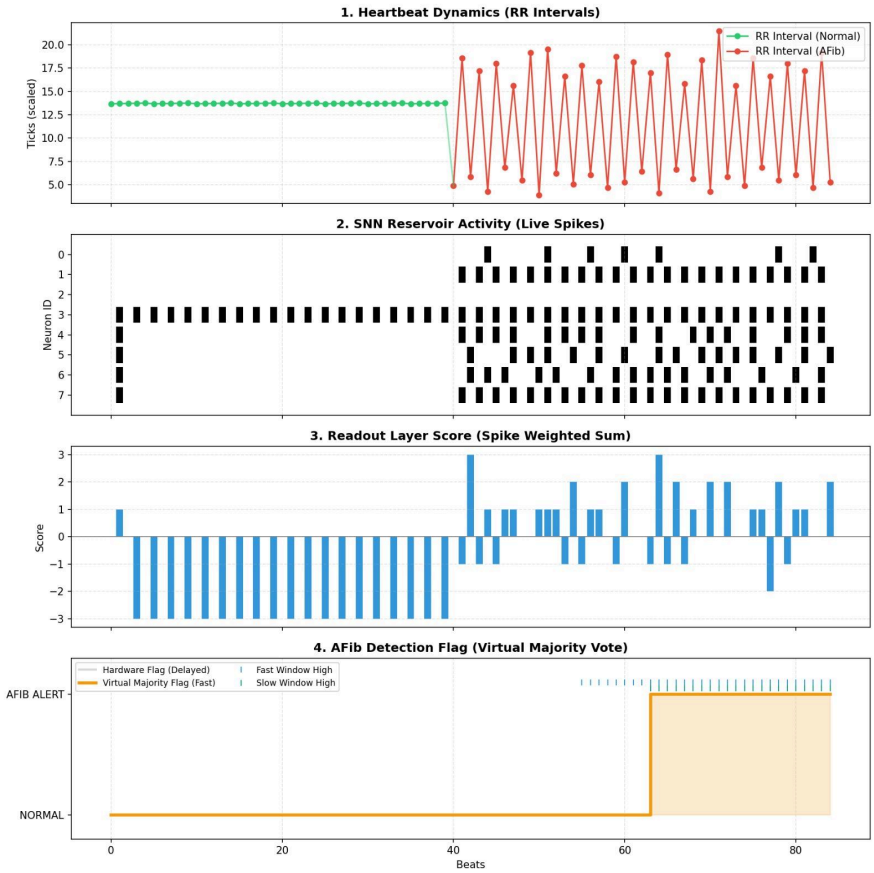


Image of functional behavior with respect to ECG Data:



- The top plot shows the time between heartbeats
- The second plot shows the spiking of the LIF neurons in the reservoir
- The third plot is the plot of the weighted sum of spikes that are displayed on the plot above
- The final plot detects the AFib flag

Future work

- This tapeout establishes the core SNN inference engine. Several natural extensions would take it from a proof-of-concept to a clinically deployable device:
- Multi-arrhythmia classification. The current design detects two conditions (AFib and asystole/bradycardia). Expanding the reservoir to 16 neurons and adding a second readout layer would enable discrimination of PVC, SVT, and atrial flutter — all rhythm-based conditions that are separable in RR-interval feature space without requiring waveform morphology.
- Clock-adaptive interval encoding. The RR interval bins are currently calibrated for the 10 kHz demo board clock (1 tick = 100 μs, 700 ms beat = 7,000 ticks). A future revision could include a one-time calibration register that scales the tick-to-interval mapping at runtime, making the chip clock-independent and deployable at any system frequency without weight retraining.

- On-chip STDP weight update. The serial weight interface already supports runtime weight updates. A future version could add a lightweight Spike-Timing Dependent Plasticity (STDP) block (~25 gates) that incrementally adjusts weights based on physician-confirmed events — enabling true patient-specific adaptation without a retraining pipeline.
- Full wearable SoC integration. The chip's event-driven architecture makes it a natural always-on front-end for a BLE SoC. Pairing it with an nRF52 or ESP32 in deep sleep — waking only on afib_flag or asystole_flag assertion — would yield a complete cardiac monitor with sub-μW average power on the detection path, suitable for multi-year coin cell operation.

Final Understanding

Overall, the testing process validates that the chip can:

- Correctly measure heartbeat intervals
- Detect irregular patterns associated with AFib
- Identify absence of heart activity (asystole)
- Operate continuously with minimal power

At the same time, it is important to recognize that this is a **prototype system**, designed to demonstrate the feasibility of ultra-low-power, hardware-based cardiac monitoring.